

BrightTutor Academic Portal

Complete End-to-End System Architecture, Business Workflows, 16-Step Verification Test Plan & Developer Handover Guide

Stack: .NET 8 Web API + Angular 19 + PostgreSQL

Architecture: Clean Architecture (CQRS + MediatR)

Environment: Backend :5198 | Frontend :4200

Status: All 16 Modules Verified & Operational



1. System Architecture & Tech Stack

BrightTutor is designed following **Clean Architecture principles** with strict separation of concerns, ensuring testability, maintainability, and loose coupling.



Backend Architecture (.NET 8)

- **BrightTutor.Domain:** Core domain entities (`User` , `Student` , `Teacher` , `Course` , `ClassGroup` , `Attendance` , `Permission`), value objects, and domain enums.
- **BrightTutor.Application:** MediatR CQRS Commands & Queries, FluentValidation pipeline behaviors, Application DTOs, and Interfaces.
- **BrightTutor.Infrastructure:** EF Core PostgreSQL DbContext, entity configurations, BCrypt password hashing, JWT Token generation, and automated seeding (`DbInitializer`).
- **BrightTutor.Api:** RESTful controllers, global exception handling, Swagger documentation, and JWT bearer authentication.



Frontend Architecture (Angular 19)

- **Standalone Components:** Modern Angular architecture with zero NgModules for faster lazy-loading and tree-shaking.
- **Reactivity & Signals:** State management powered by Angular Signals (`signal` , `computed`) and reactive data fetching (`rxResource`).
- **Dynamic Searchable Selects:** Reusable `SearchableSelectComponent` with instant typeahead search across thousands of entities (no raw GUID inputs).
- **Vanilla CSS Design System:** Clean styling, glassmorphism, responsive data cards, status badges, and subtle micro-animations.



2. Role-Based Access Control & Permission Matrix

The system implements a hybrid **Role-Based (RBAC) + Dynamic Permission System**. Permissions are stored in the PostgreSQL database and can be dynamically customized per role by the Super Admin at runtime without code deployment.

Permission Code	Module	Description	Admin	Teacher	Student	Parent
<code>users.view</code> / <code>manage</code>	Users	Create, edit, deactivate, and assign user roles	✓ Full	✗	✗	✗
<code>courses.view</code> / <code>manage</code>	Courses	Manage courses, subjects, class groups & capacities	✓ Full	✗	✗	✗
<code>attendance.mark</code>	Attendance	Submit Group, Online, and Home Visit attendance	✓ All	✓ Own	✗	✗
<code>attendance.view</code>	Reports	View attendance logs, calendars, and performance summaries	✓ All	✓ Class/Own	✓ Personal Only	✓ Child Only
<code>settings.manage</code>	Settings	Dynamic Permission Matrix & Admin System Actions	✓ Full	✗	✗	✗

🔒 Critical Security & Privacy Invariants

- **Super Admin Immunity:** The account `admin@brighttutor.com` can NEVER be deactivated and can only be edited by themselves.
- **Teacher ID Locking:** In attendance marking forms and teacher reports, the Teacher ID is permanently locked to the logged-in teacher's ID. Teachers cannot submit attendance on behalf of other instructors.
- **Student Privacy Scoping:** Students only see their own attendance records on the calendar, summary, and group attendance views. Other students' records in the class are filtered out.

3. Pre-Seeded Developer Accounts & Credentials

Super Admin Account

Email: `admin@brighttutor.com`

Password: `AdminPass123!`

Scope: Full system access, users management, dynamic matrix, all settings & reports.

Teacher Account

Email: `sarah.jenkins@brighttutor.com`

Password: `TeacherPass123!`

Scope: Group, Online & Home attendance marking, class reports, personal stats.

Student Account

Email: `michael.brown@brighttutor.com`

Password: `StudentPass123!`

Scope: Personal calendar, attendance summary, own records only.

4. Complete 16-Step End-to-End Test Journey (User Creation to Deployment)

Follow these 16 sequential steps to test the entire BrightTutor lifecycle from scratch.

STEP 1: Super Admin Authentication & Sign-In

Admin (Super Admin)

Action: Navigate to `/login` and sign in with `admin@brighttutor.com` / `AdminPass123!`.

Verification: Issues JWT token, saves session to `localStorage`, displays topbar badge `Super Admin`  `Super Admin`, and redirects to Dashboard.

STEP 2: Executive Dashboard Metrics & Monitoring

Admin (Super Admin)

Action: Navigate to `/dashboard`.

Verification: Displays 4 real-time summary cards (*Total System Users, Active Students, Registered Teachers, Active Courses*) and Today's Attendance Real-Time Summary.

STEP 3: Users Management, Account Creation, Edit & Deactivation

Admin (Super Admin)

Action: Open `/users`. Create Teacher `sarah.jenkins@brighttutor.com` and Student `michael.brown@brighttutor.com`. Test Edit () and Deactivate () actions.

Verification: Modal saves without page reload. Super Admin is protected from deactivation. Bottom error/success alerts display properly.

STEP 4: Courses & Class Groups Management

Admin (Super Admin)

Action: Open </courses> . Create Course (e.g. *Mathematics 101*) and create Class Group (*Section A - Morning*, Capacity: 30). Assign Teacher.

Verification: Group is created and immediately populated into all attendance searchable dropdowns.

STEP 5: Group Attendance Marking & UPSERT Submission

Admin / Teacher

Action: Open </mark-group-attendance> . Select *Section A - Morning*. Use **1-Click Bulk Mark (✓ Mark All Present)**, adjust individual status buttons, and Submit.

Verification: Database executes UPSERT query (no duplicate key errors if submitted repeatedly for the same date).

STEP 6: View Group Attendance Records

Admin / Teacher

Action: Open </view-group-attendance> . Select *Section A - Morning* and session date.

Verification: Auto-loads attendance table showing Student Name, Class Group, Teacher, and green **Present** status badges.

STEP 7: Staff Teacher Attendance Marking

Admin / Supervisor

Action: Open `/mark-teacher-attendance` . Select Teacher `sara jenkins` and mark present.

Verification: Submits supervisor-verified staff attendance record to `TeacherAttendances` table.

STEP 8: Online Session Attendance Marking

Teacher

Action: Open `/mark-online-attendance` . Select Student `Michael brown` , group, and add session notes (e.g. *Completed 60-min Zoom session on Algebra*).

Verification: Records online tutoring session with timestamp and notes.

STEP 9: Home Visit Tutoring GPS Check-In & Check-Out

Teacher

Action: Open `/home-attendance` . Perform **Check-In** with GPS coordinates and address. Next, select the active visit session from the dropdown and submit **Check-Out**.

Verification: Records GPS location verification, active session links automatically, and records duration on checkout.

STEP 10: Class Attendance Reports & Student Breakdown

Admin / Teacher

Action: Open `/class-report` . Select *Section A - Morning*.

Verification: Displays class summary totals (Total Records, Present, Absent, Late, Excused, Rate %) and per-student breakdown table showing `Michael brown | 100%` .

STEP 11: Teacher Reports & Student Calendar

Teacher / Student

Action: Open `/teacher-report` (shows Sara's conducted classes) and `/student-calendar` (shows Michael's monthly attendance dates).

Verification: Auto-resolves foreign key IDs and displays monthly dates with color-coded status badges.

STEP 12: Dynamic Role-Permission Matrix & Admin Settings

Admin (Super Admin)

Action: Open `/permissions` . Toggle permissions for Teacher or Student roles and click *Save Role Permissions*.

Verification: Persisted directly to PostgreSQL `RolePermissions` table; dynamically updates live user navigation.

STEP 13: Student Portal Verification

Student (Michael brown)

Action: Sign in as `michael.brown@brighttutor.com` / `StudentPass123!` .

Verification: Topbar displays `Michael brown`  `Student` . Sidebar shows only student menus. Calendar and Summary **automatically lock to Michael** and display his 100% attendance records.

STEP 14: Teacher Portal Verification

Teacher (sara jenkins)

Action: Sign in as `sarah.jenkins@brighttutor.com` / `TeacherPass123!` .

Verification: Topbar displays `sara jenkins`  `Teacher` . Admin menus (*Users, Matrix, Settings, Courses, Teacher Attendance*) are hidden. Teacher selector is locked to Sara.

STEP 15: Teacher Student Lookup Authorization Fix

Teacher

Action: Open Group Attendance and Online Attendance as Teacher.

Verification: `[Authorize(Roles = "Admin,Teacher")]` on `GET /api/users` allows Teacher to load enrolled students (`Michael brown`) without 403 Forbidden errors.

STEP 16: Account Profile Badge & Change Password Modal

All Roles

Action: Click topbar user badge ( `Profile & Password`). Enter current password and new password. Click *Update Password*.

Verification: Backend updates BCrypt hash via `/api/auth/change-password` ; displays floating green toast `✓ Password updated successfully!` .

5. REST API Endpoints & Entity Reference

HTTP Method	Endpoint URI	Authorization	Functionality
POST	/api/auth/login	Anonymous	Authenticates user and returns JWT Token & User Info
POST	/api/auth/change-password	[Authorize]	Changes current authenticated user's password
GET / POST	/api/users	[Authorize(Roles="Admin,Teacher")]	Lists active users / Registers new users (Admin only)
GET / POST	/api/courses	[Authorize]	Retrieves course catalog / Creates courses
GET / POST	/api/classgroups	[Authorize]	Retrieves class groups / Creates groups with capacity
POST	/api/attendance/group	[Authorize(Roles="Admin,Teacher")]	Submits roster attendance using UPSERT strategy
GET	/api/attendance/group	[Authorize]	Retrieves group attendance records for date
POST	/api/attendance/online	[Authorize(Roles="Admin,Teacher")]	Records online session attendance
POST	/api/attendance/home/check-in	[Authorize(Roles="Admin,Teacher")]	GPS Check-In for student home visit
POST	/api/attendance/home/check-out	[Authorize(Roles="Admin,Teacher")]	GPS Check-Out for student home visit

HTTP Method	Endpoint URI	Authorization	Functionality
GET	/api/attendance/class-report	[Authorize]	Aggregate attendance metrics for class group
GET	/api/attendance/teacher-report	[Authorize]	Conducted session statistics for teacher
GET	/api/attendance/student-calendar	[Authorize]	Monthly calendar attendance days for student
GET	/api/attendance/student-summary	[Authorize]	Aggregate performance rate for student
GET / POST	/api/permissions/roles/{roleId}	[Authorize]	Fetches / Updates dynamic role permissions
GET	/api/dashboard/summary	[Authorize(Roles="Admin")]	Executive dashboard system metrics

💡 How to Run the Solution Locally

1. **Backend:** Navigate to `backend/` and run `dotnet run --project BrightTutor.Api` (Starts on `http://localhost:5198`).
2. **Frontend:** Navigate to `frontend/brightttutor-client/` and run `npm start` or `ng serve` (Starts on `http://localhost:4200`).
3. **Database:** PostgreSQL running at `localhost:5432` , database `brightttutor` . All migrations and seed data auto-apply on startup via `DbInitializer.SeedAsync()` .